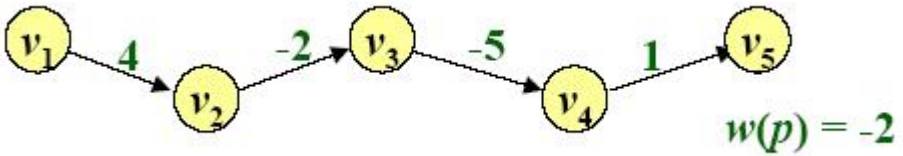
Chapter24 单源最短路径

- 给出一个带权有向图$G=(V,E)$, 加权函数是从边到实数权值的映射$w: E \rightarrow R$. 路径 $p=v_1 \rightarrow v_2 \text{--} \ldots \rightarrow v_k$ 的权是指其组成边的所有权值之和

  $w(p)=\Sigma_{i=1..k-1} w(v_i, v_{i+1})$

Example:



- 从 u 到 v 的最短路径的权是:

$$\delta(u,v)=\begin{cases}\min\{w(p): \text{if there is a path } p \text{ from } u \text{ to } v\} \\ \infty \qquad\qquad \text{otherwise}\end{cases}$$

- *单源最短路径问题：*给出一个带权有向图*G=(V,E)*,找出从某个给定源顶点s到每个顶点v的最短路径。

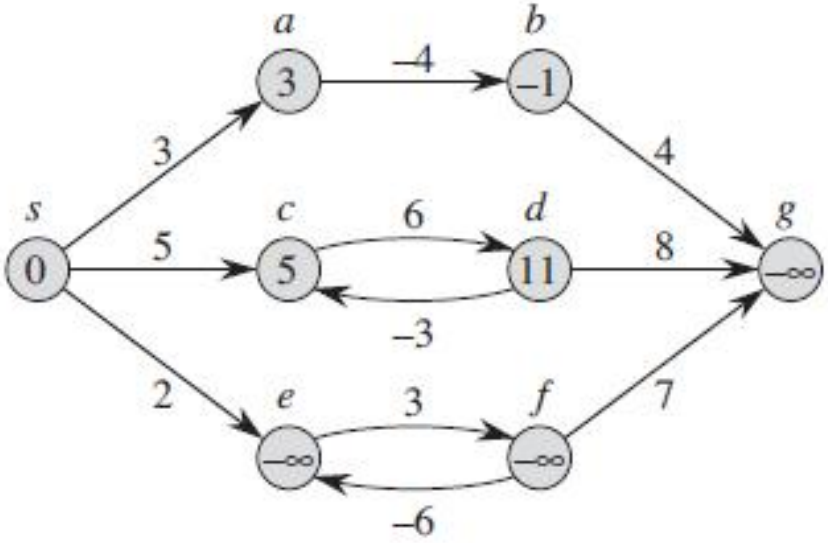
- *单源最短路径的变体：*
  1. *单终点最短路径问题*
  2. *单对顶点最短路径问题*
  3. *每对顶点最短路径问题*

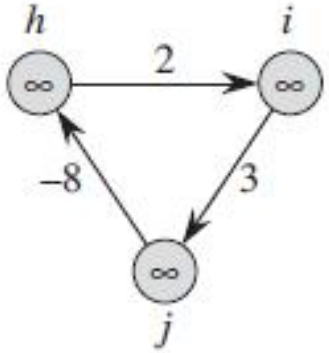
•最短路径的最优子结构

引理 24.1(最短路径的子路径是最短路径) 对于一给定的带权有向图 $G=(V, E)$，所定义的权函数为 $w: E \to \mathbf{R}$。设 $p=\langle v_1, v_2, \cdots, v_k\rangle$是从 v_1 到 v_k 的最短路径。对于任意 i，j，其中 $1 \leqslant i \leqslant j \leqslant k$，设 $p_{ij}=\langle v_i, v_{i+1}, \cdots, v_j\rangle$为 p 中从顶点 v_i 到顶点 v_j 的子路径。那么，p_{ij} 是从 v_i 到 v_j 的最短路径。

•负权值边

给出一个带权有向图*G=(V,E)*,可能存在权值为负值的边。如果图中不包含从源s可达的负权回路，则对于所有顶点u和v,最短路径的权的定义*δ(u,x)*依然正确。如果顶点u和v存在一条负权回路，就定义*δ(u,x)*=-∞





•回路

一条最短路径不包含回路。图*G=(V,E)*中任意一条无环路径包含至多丨V丨个不同的顶点，同时包含至多丨V丨-1条边。

•最短路径的表示

已知图*G=(V,E)*中，对每个顶点v，设置其前趋顶点π[v]为另一顶点或为NIL。最短路径算法设置每个顶点的前趋顶点，得到从s到v的最短路径。

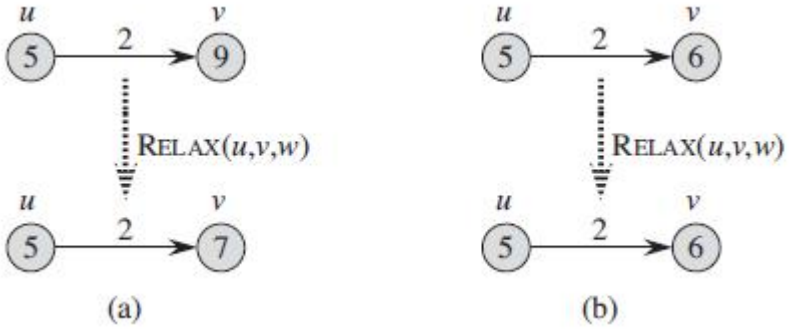
最短路径问题

松弛技术

本章的算法使用了松弛(relaxation)技术。对每个顶点 $v \in V$，都设置一个属性 $d[v]$，用来描述从源点 s 到 v 的最短路径上权值的上界，称为最短路径估计(shortest-path estimate)。

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v∈V[G]
2        do d[v] ← ∞
3           π[v] ← NIL
4  d[s] ← 0
```

```
RELAX(u, v, w)
1  if d[v]>d[u]+w(u, v)
2     then d[v] ← d[u]+w(u, v)
3          π[v] ← u
```



最短路径问题

三角不等式(引理 24.10)

对任意边$(u, v)\in E$，有$\delta(s, v)\leqslant\delta(s, u)+w(u, v)$

上界性质(引理 24.11)

对任意顶点$v\in V$，有$d[v]\geqslant\delta(s, v)$，而且一旦$d[v]$达到$\delta(s, v)$值就不再改变。

无路径性质(推论 24.12)

如果从s到v不存在路径，则总是有$d[v]=\delta(s, v)=\infty$。

收敛性质(引理 24.14)

如果$s\rightsquigarrow u\rightarrow v$是图$G$某$u, v\in V$的最短路径，而且在松弛边$(u, v)$之前的任何时间$d[u]=\delta(s, u)$，则在操作过后总有$d[v]=\delta(s, v)$。

路径松弛性质(引理 24.15)

如果$p=\langle v_0, v_1, \cdots, v_k\rangle$是从$s=v_0$到$v_k$的最短路径，而且$p$的边按照$(v_0, v_1)(v_1, v_2), \cdots,(v_{k-1}, v_k)$的顺序进行松弛，那么$d[v_k]=\delta(s, v_k)$。这个性质的保持并不受其他松弛操作的影响，即使它们与p的边上的松弛操作混合在一起也是一样的。

前趋子图性质(引理 24.17)

一旦对于所有$v\in V$，$d[v]=\delta(s, v)$，前趋子图就是一个以s为根的最短路径树。

Bellman-Ford算法

- Bellman-Ford算法能在一般的情况（存在负权边的情况）下，解决单源最短路径问题。给出一个带权有向图$G=(V,E)$,其源点为s，加权函数为$w:E \rightarrow R$,对该图运行Bellman-Ford算法后可以返回一个布尔值，表明图中是否存在着一个从源点可达的权为负的回路。

- 基本思想：运用松弛技术，对每个顶点v，逐步减小从源s到v的最短路径的权的估计值d[v]直到其达到实际最短路径的权$\delta(s,v)$

Bellman-Ford算法

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i ← 1 to |V[G]| −1
3        do for each edge (u, v) ∈ E[G]
4              do RELAX(u, v, w)
5  for each edge(u, v) ∈ E[G]
6        do if d[v] > d[u] + w(u, v)
7              then return FALSE
8  return TRUE
```

Bellman-Ford 算法的运行时间为 $O(VE)$，因为第 1 行的初始化占用时间为 $\Theta(V)$，第 2～4 行对边进行 $|V|-1$ 趟操作，每趟操作的运行时间为 $\Theta(E)$。第 5～7 行的 **for** 循环运行时间为 $O(E)$。

Bellman-Ford算法

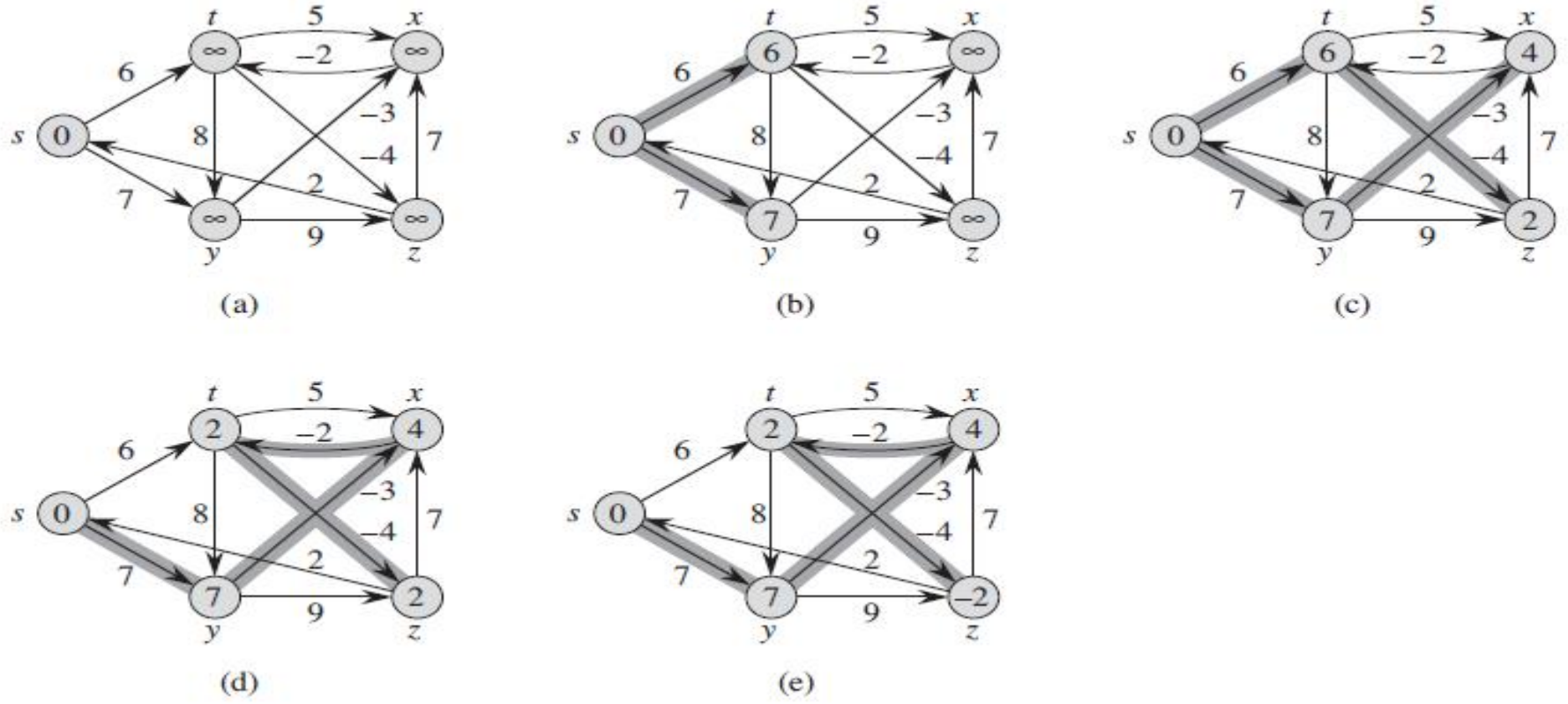
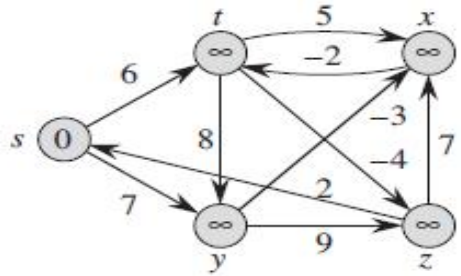
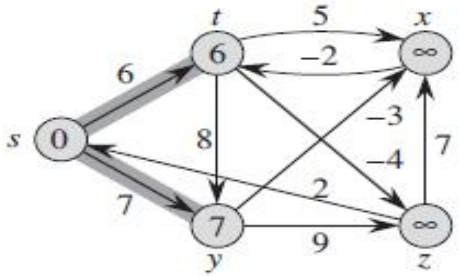


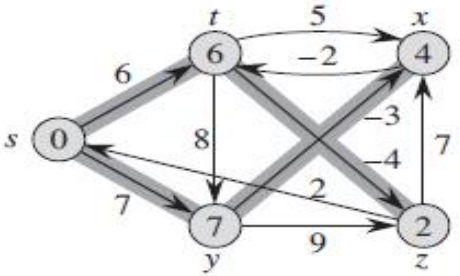
图 24-4 Bellman-Ford 算法的执行过程。源点是顶点 s。d 值被标记在顶点内，阴影覆盖的边指示了前趋值：如果边(u, v)被阴影覆盖，则 $\pi[v]=u$。在这个特定的例子中，每一趟按照如下的顺序对边进行松弛：(t, x)，(t, y)，(t, z)，(x, t)，(y, x)，(y, z)，(z, x)，(z, s)，(s, t)，(s, y)。a)示出了对边进行第一趟操作前的情况。b)至 e)示出了每一趟连续对边操作后的情况。e)中 d 和 π 值是最终结果。Bellman-Ford 算法在这个例子中返回的是 TRUE



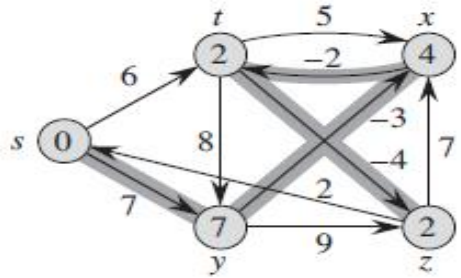
(a)



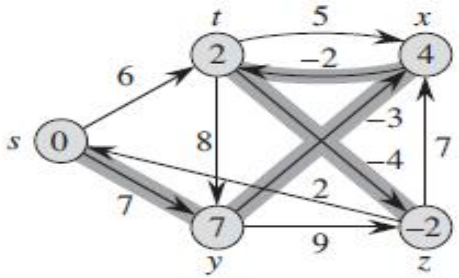
(b)



(c)



(d)



(e)

路径松弛性质(引理 24.15)

如果 $p=\langle v_0, v_1, \cdots, v_k\rangle$ 是从 $s=v_0$ 到 v_k 的最短路径，而且 p 的边按照 $(v_0, v_1)(v_1, v_2), \cdots, (v_{k-1}, v_k)$ 的顺序进行松弛，那么 $d[v_k]=\delta(s, v_k)$。这个性质的保持并不受其他松弛操作的影响，即使它们与 p 的边上的松弛操作混合在一起也是一样的。

第1轮 $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$

第2轮 $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$

第3轮 $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$

第4轮 $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$

Bellman-Ford算法

引理 24.2 设 $G=(V, E)$为带权有向图，其源点为 s，权函数为 $w: E \to \mathbf{R}$，并且假定 G 中不包含从 s 点可达的负权回路。那么，在 BELLMAN-FORD 第 2～4 行 **for** 循环的 $|V|-1$ 次迭代后，对任何 s 可达的顶点 v，有 $d[v]=\delta(s, v)$。

证明：证明此引理将使用路径松弛性质。设 v 为从 s 可达的任意顶点，$p=\langle v_0, v_1, \cdots, v_k\rangle$ 是任一条从 s 到 v 的无环最短路径，其中 $v_0=s$ 和 $v_k=v$。路径 p 有至多 $|V|-1$ 条边，所以 $k \leqslant |V|-1$。第 2～4 行的 **for** 循环的 $|V|-1$ 次迭代，其每一次都松弛了所有 E 边。第 i 次迭代被松弛的边为(v_{i-1}, v_i)，其中 $i=1, 2, \cdots, k$。由路径松弛性质，有 $d[v]=d[v_k]=\delta(s, v_k)=\delta(s, v)$。 ■

定理 24.4(Bellman-Ford 算法的正确性) 设 $G=(V, E)$为带权有向图，源点为 s，权函数为 $w: E \to \mathbf{R}$，对该图运行 Bellman-Ford 算法。若 G 不包含 s 可达的负权回路，则算法返回 TRUE，对所有顶点 $v \in V$，有 $d[v]=\delta(s, v)$成立。前趋子图 G_π 是以 s 为根的最短路径树。如果 G 包含从 s 可达的负权回路，则算法返回 FALSE。

Bellman-Ford算法

动态规划

$dp(i,v)$ 表示最多使用 i 条边的从 s 到 v 的最短路径。

- Case 1: 从 s 到 v最多使用 $i-1$ 条边

  $dp(i,v) = dp(i-1,v)$
- Case 2: 从 s 到 v恰好使用 i 条边

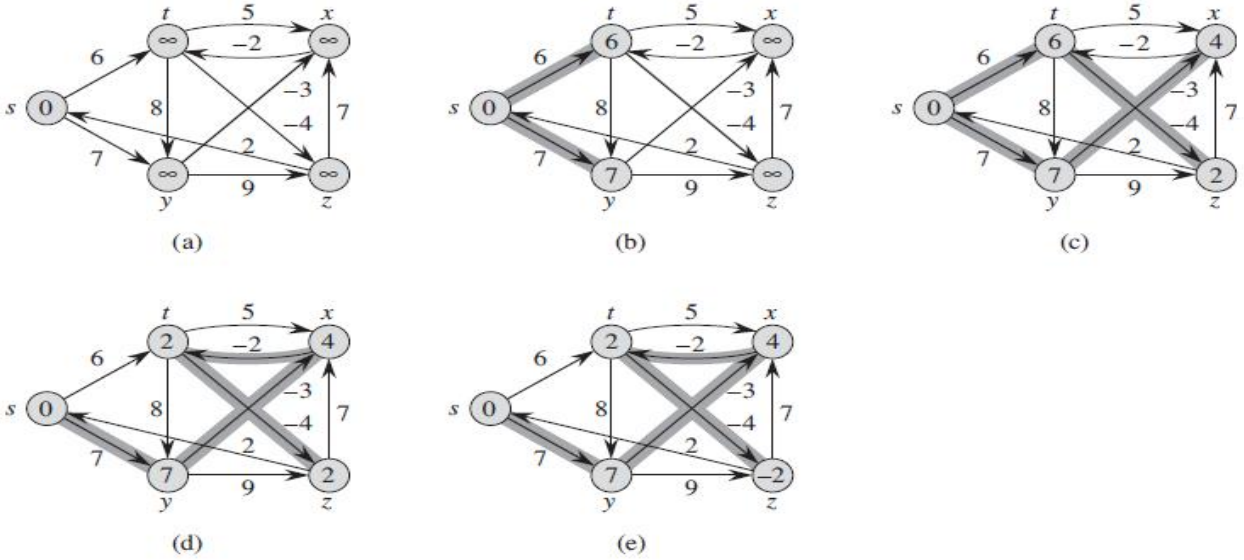
  $dp(i,v) = \min\limits_{(u,v)\in E}\{dp(i-1,u) + cost_{uv}\}$

所以，递推关系式如下：

$$dp(i,v) = \begin{cases} \infty & i = 0 \\ \min\{dp(i-1,v), \min\limits_{(u,v)\in E}\{dp(i-1,u) + cost_{uv}\}\} & otherwise \end{cases}$$

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i ← 1 to |V[G]| − 1
3        do for each edge (u, v) ∈ E[G]
4              do RELAX(u, v, w)
5  for each edge(u, v) ∈ E[G]
6        do if d[v] > d[u] + w(u, v)
7              then return FALSE
8  return TRUE
```

```
RELAX(u, v, w)
1  if d[v] > d[u] + w(u, v)
2     then d[v] ← d[u] + w(u, v)
3          π[v] ← u
```



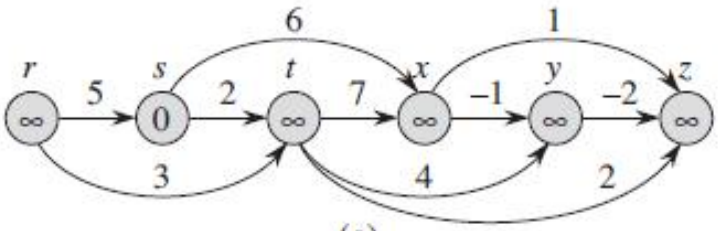
有向无回路图中的单源最短路径

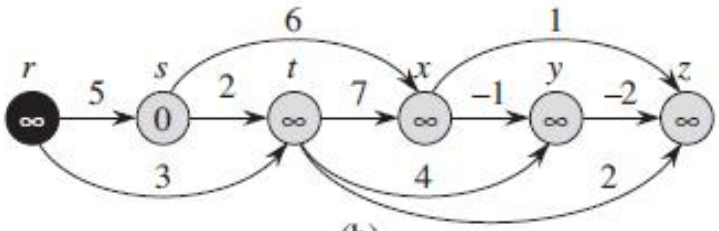
```
DAG-SHORTEST-PATHS(G, w, s)
1  topologically sort the vertices of G
2  INITIALIZE-SINGLE-SOURCE(G, s)
3  for each vertex u, taken in topologically sorted order
4       do for each vertex v ∈ Adj[u]
5            do RELAX(u, v, w)
```

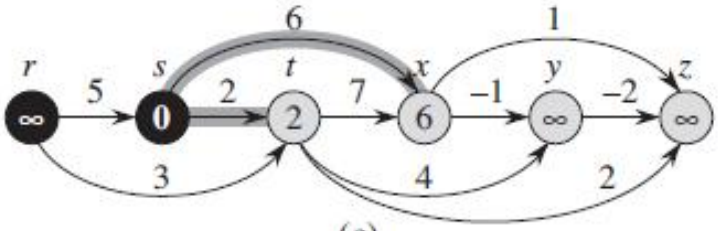
此算法的运行时间很容易分析。如 22.4 节所示，第 1 行的拓扑排序需 $\Theta(V+E)$时间。第 2 行调用的 INITIALIZE-SINGLE-SOURCE 需 $\Theta(V)$时间。第 3～5 行的 for 循环中对每一个顶点有一次迭代。对每一个顶点，其每条出边都只被检测过一次。第 4～5 行的 for 循环内部总共有 $|E|$ 次迭代(我们在这里使用了聚集分析)。因为 for 循环内部的每次迭代需 $\Theta(1)$时间，总的运行时间为 $\Theta(V+E)$，这与图的邻接表表示的大小成线性关系。

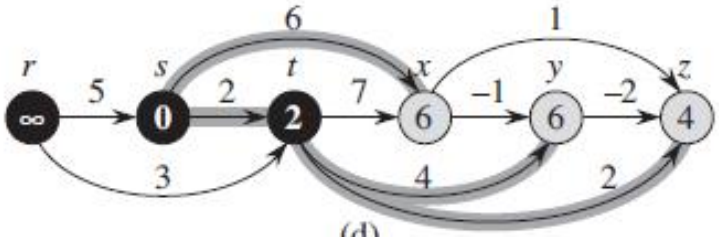
对一个有向无环图(Directed Acyclic Graph简称DAG)G进行拓扑排序，是将G中所有顶点排成一个线性序列，使得图中任意一对顶点u和v，若边(u,v)∈E(G)，则u在线性序列中出现在v之前。

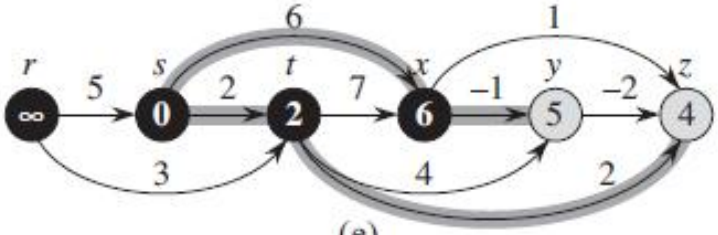
有

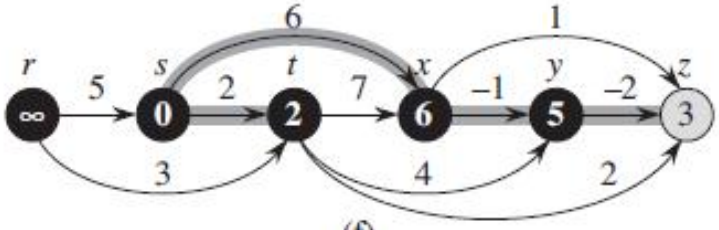

r s t x y z
∞ 0 ∞ ∞ ∞ ∞
5 6 2 7 −1 1 −2
3 4 2
(a)

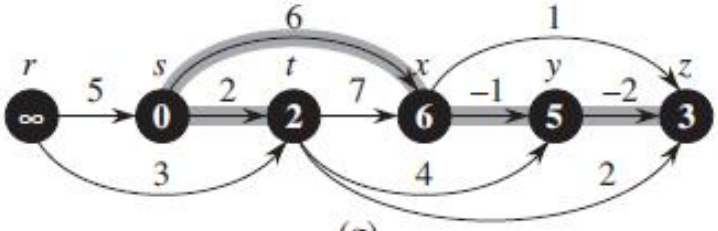

r s t x y z
∞ 0 ∞ ∞ ∞ ∞
5 6 2 7 −1 1 −2
3 4 2
(b)


r s t x y z
∞ 0 2 6 ∞ ∞
5 6 2 7 −1 1 −2
3 4 2
(c)


r s t x y z
∞ 0 2 6 6 4
5 6 2 7 −1 1 −2
3 4 2
(d)


r s t x y z
∞ 0 2 6 5 4
5 6 2 7 −1 1 −2
3 4 2
(e)


r s t x y z
∞ 0 2 6 5 3
5 6 2 7 −1 1 −2
3 4 2
(f)


r s t x y z
∞ 0 2 6 5 3
5 6 2 7 −1 1 −2
3 4 2
(g)

有向无回路图中的单源最短路径

定理 24.5 如果一个带权有向图 $G=(V, E)$ 有源点 s 而且无回路，则在 DAG-SHORTEST-PATHS 终止时，对任意顶点 $v \in V$，有 $d[v]=\delta(s, v)$，且前趋子图 G_π 是最短路径树。

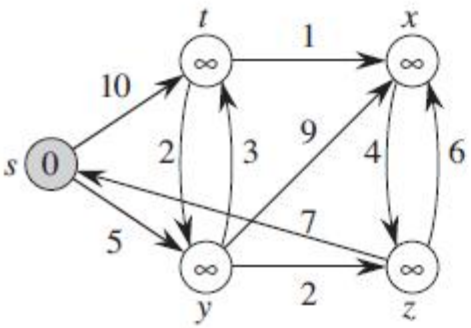
证明：我们首先来证明算法结束时，对所有顶点 $v \in V$，有 $d[v]=\delta(s, v)$。如果 v 是从 s 不可达的，根据无路径性质 $d[v]=\delta(s, v)=\infty$。现在假设 v 从 s 可达，因此存在一条最短路径 $p=\langle v_0, v_1, \cdots, v_k\rangle$，其中 $v_0=s$ 和 $v_k=v$。因为我们是依据拓扑排序对顶点进行处理的，所以 p 上的边被松弛的顺序为 $(v_0, v_1), (v_1, v_2), \cdots, (v_{k-1}, v_k)$。路径松弛性质暗示在终止时，$d[v_i]=\delta(s, v_i)$，其中 $i=0, 1, \cdots, k$。最后，根据前趋子图性质，G_π 是一棵最短路径树。 ■

Dijkstra算法

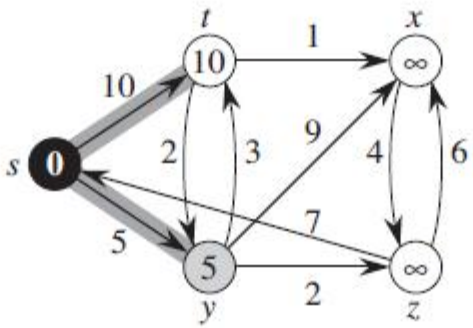
- 基本思想：
  1. 设置一个顶点集合S，从源点s到集合中的顶点的最短路径的权值已确定
  2. 算法反复选择具有最短路估计的顶点 $u \in V-S$，并将u加入到S中.
  3. 对u的所有出边进行松弛.

Dijkstra算法

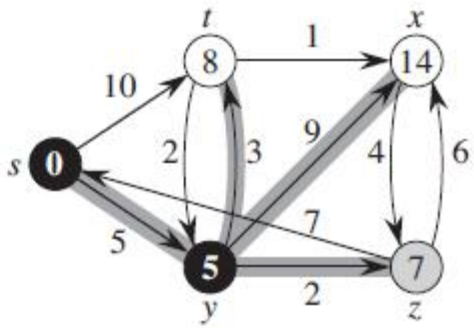
```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S ← ∅
3  Q ← V[G]
4  while Q≠∅
5        do u ← EXTRACT-MIN(Q)
6           S ← S∪{u}
7           for each vertex v∈Adj[u]
8                 do RELAX(u, v, w)
```



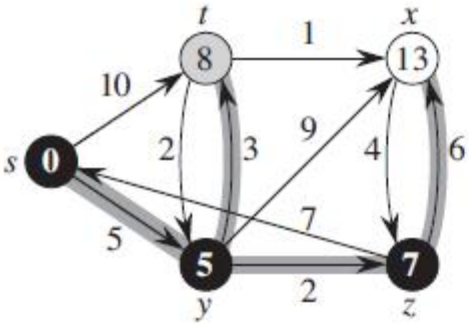
(a)



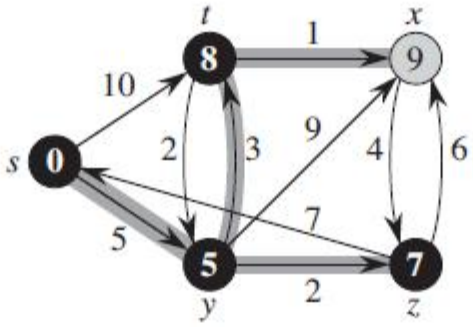
(b)



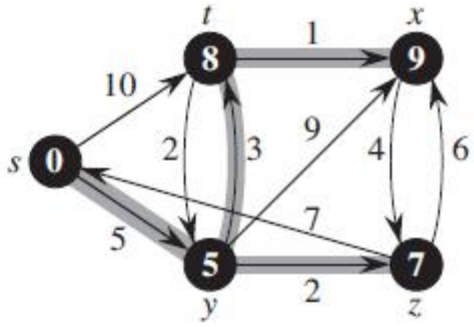
(c)



(d)



(e)



(f)

Dijkstra算法运行复杂度分析

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S ← ∅
3  Q ← V[G]
4  while Q≠∅
5       do u ← EXTRACT-MIN(Q)
6          S ← S∪{u}
7          for each vertex v∈Adj[u]
8               do RELAX(u, v, w)
```

Dijkstra通过调用三种优先队列操作维护最小优先队列Q：

- INSERT，调用次数|V|
- EXTRACT-MIN，调用次数|V|
- DECREASE-KEY, 调用次数|E|

Dijkstra运行时间依赖于最小优先队列的具体实现

（1）采用数组实现：$O(V^2+E)$

- INSERT，O(1)
- EXTRACT-MIN，O(V)
- DECREASE-KEY, O(1)|

（2）采用二叉最小堆实现：O((V+E)lgV)

- 构建堆：O(V)
- EXTRACT-MIN，O(lgV)
- DECREASE-KEY, O(lgV)|

（3）采用斐波那契最小堆实现：O(VlgV+E)

- 构建堆：O(V)
- EXTRACT-MIN，O(lgV)
- DECREASE-KEY, O(1)|

课堂练习24.1

设G是一个n个顶点和e条边的带权有向图，各边的权值为0到N-1之间的整数，N为一非负整数。实现Dijkstra算法，使其能在O(Nn+e)时间内计算出从源到所有其他顶点之间的最短路径长度。